CS445 - BASICS OF WEB PROGRAMMING

NOTES - LAB ASSIGNMENT 7 | JSON AND XML - CAMPUS EVENT BOARD

Team Details

Role	Student name	Roll no.
Team Leader	Hanvate Chandrakant Devidas	40
Team Member	Muhammad Faizan	56
Team Member	Mohd Tahseen Raza	55
Team Member	Akash Chaman	08
Team Member	Ramesh Kumar Bhagat	71

Q1. Show the event table and write one event as JSON and XML. Point out three correspondences.

id	title	date (YYYY-MM-DD)	seats	free
1	HACER 2025 Conference	2025-12-19	0	true
2	7th Inter IIIT Sports Meet	2025-03-20	12	true
3	Reduce E-Waste Recognition	2025-06-05	4	true

JSON (events.json)

```
{
"id": 1,
"title": "HACER 2025 Conference",
"date": "2025-12-19",
"seats": 0,
"free": true
}
```

XML (events.xml)

```
<event id="1">
<title>HACER 2025 Conference</title>
<date>2025-12-19</date>
<seats>0</seats>
<free>true</free>
</event>
```

Three correspondences: title column -> "title" key -> <title> element; date column -> "date" key -> <date> element; seats column -> "seats" key -> <seats> element.

Q2. Which JSON value types did you use? Give one value of each.

String: "title": "HACER 2025 Conference". Number: "seats": 0. Boolean: "free": true.

Q3. Why must a JSON key be in double quotes? Give one line valid in JavaScript but invalid in JSON.

JSON has a fixed syntax, so keys must be in double quotes. JavaScript allows an unquoted key and a single-quoted string, but JSON does not.

```
const person = { name: 'Akash' };
```

Q4. In events.xml, which field is an attribute and which are elements? Why?

The id is an attribute, written as <event id="1">. The title, date, seats, and free fields are child elements. I used id as an attribute because it is a short identifier. The other fields are elements because they contain event details.

Q5. Paste the validator error from the broken JSON and the fix.

I added a comma after the last event object by mistake. JSONLint showed this error:

```
Invalid JSON E008: trailing comma in array line 41, col 1
```

Broken ending:

```
        },
    ]
```

I removed the extra comma. The corrected ending is:

```
        }
    ]
```

After the correction, JSONLint showed Valid JSON. The error and fixed JSON screenshots are saved as 05_C1_JSON_Error.png and 06_C1_JSON_Fixed.png in Output Screenshort/LAB 7/.

Q6. How did you read one value in JSON and XML? Paste both lines.

JSON - array index and dot access:

```
console.log('First event title:', events[0].title);
console.log('Last event seats:', events[events.length - 1].seats);
```

XML - get the first title:

```
const titles = xmlDocument.getElementsByTagName('title');
const firstTitle = titles[0].textContent;
```

Q7. What are the byte sizes of events.json and events.xml? Why is XML larger?

File	Size
events.json	1,564 bytes
events.xml	1,785 bytes

XML is 221 bytes larger in this project because it has opening and closing tags around the values.

Q8. Neither format has a real date type. Which convention did you use, and why is it unambiguous?

I used ISO-8601 format YYYY-MM-DD, for example 2025-12-19. The year-month-day order is clear and does not confuse day and month, so it can be understood consistently in different countries. In JSON the date is a string. XML also stores the value as text unless another schema or application converts it.

Q9. List your three schema rules. Why is it important to validate data before your code uses it?

1. Every event must have a non-empty title.
2. Every date must follow YYYY-MM-DD format.
3. Seats must be a whole number greater than or equal to zero.

Validation helps find missing or incorrect values before the website uses them. It prevents wrong information from appearing on event cards and avoids errors in the program.

Q10. Which format did you choose for the page, and why? Give one situation where XML is better. Paste the GitHub repository link.

I chose JSON for the Event Board because the website uses JavaScript. JavaScript can load JSON with fetch() and response.json(), and the data can be used directly to create event cards. XML is a better choice when a system or web service exchanges structured XML messages, for example a SOAP web service.

GitHub repository link: https://github.com/Akash0409/Campus-Event-Board